ARCHITECTURE & SYSTEM DESIGN REFERENCE

KAVACH

Complete System Architecture Diagrams

ET AI Hackathon 2026 · Problem Statement 6
AI for Digital Public Safety

DIAGRAM 01

Top-Level System Architecture

Three-layer architecture: Citizen & Analyst clients → FastAPI Gateway → Deterministic AI Engine + Intelligence modules + Data Persistence.

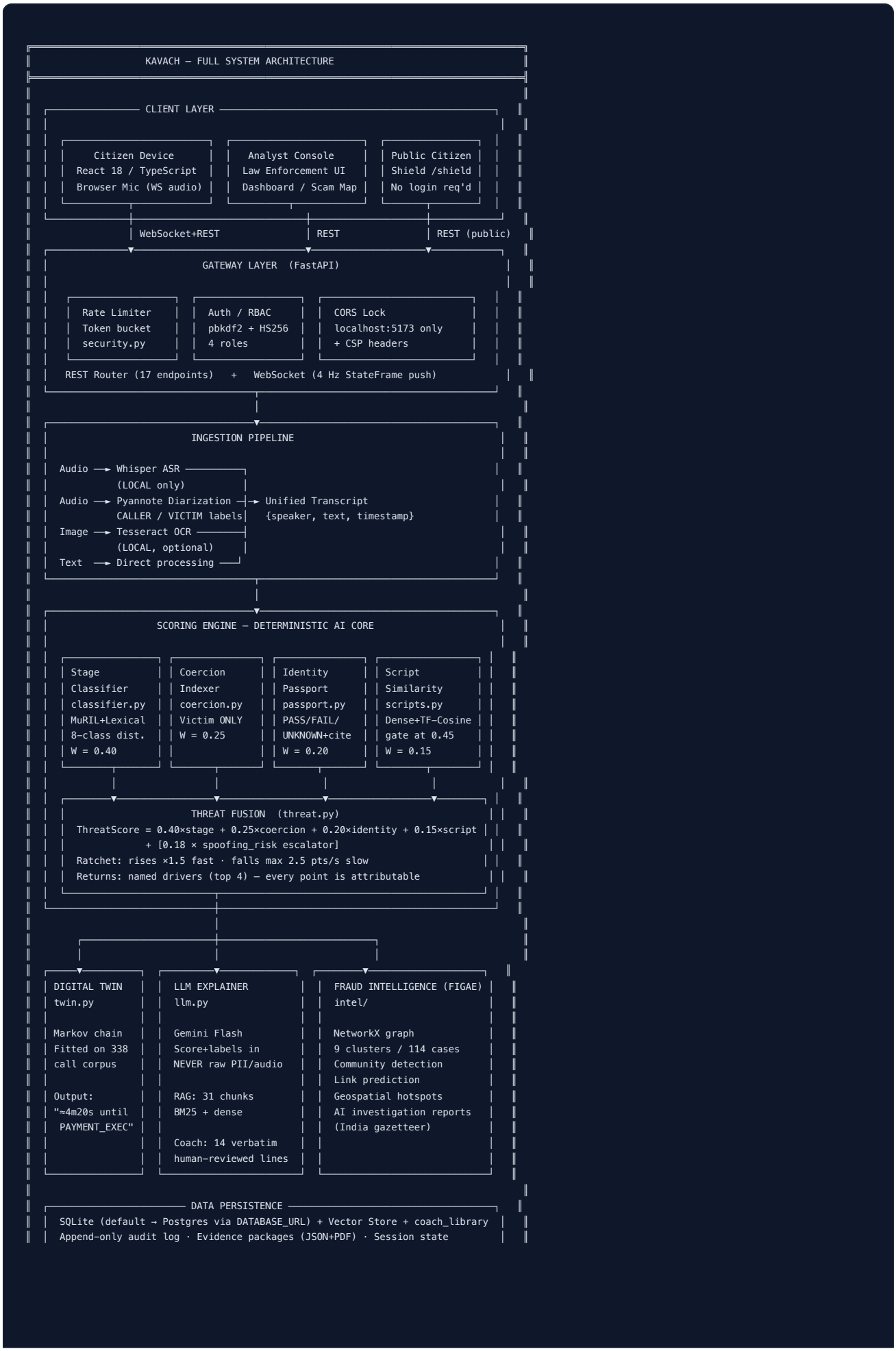
KAVACH — FULL SYSTEM ARCHITECTURE
CLIENT LAYER
Citizen Device
React 18 / TypeScript
Browser Mic (WS audio)
Analyst Console
Law Enforcement UI
Dashboard / Scam Map
Public Citizen
Shield /shield
No login req'd
WebSocket+REST
REST
REST (public)
GATEWAY LAYER (FastAPI)
Rate Limiter
Token bucket
security.py
Auth / RBAC
pbkdf2 + HS256
4 roles
CORS Lock
localhost:5173 only
+ CSP headers
REST Router (17 endpoints) + WebSocket (4 Hz StateFrame push)
INGESTION PIPELINE
Audio → Whisper ASR
(LOCAL only)
Audio → Pyannote Diarization
CALLER / VICTIM labels
Unified Transcript
{speaker, text, timestamp}
Image → Tesseract OCR
(LOCAL, optional)
Text → Direct processing
SCORING ENGINE — DETERMINISTIC AI CORE
Stage Classifier
classifier.py
MuRIL+Lexical
8-class dist.
W = 0.40
Coercion Indexer
coercion.py
Victim ONLY
W = 0.25
Identity Passport
passport.py
PASS/FAIL/
UNKNOWN+cite
W = 0.20
Script Similarity
scripts.py
Dense+TF-Cosine
gate at 0.45
W = 0.15
THREAT FUSION (threat.py)
ThreatScore = 0.40×stage + 0.25×coercion + 0.20×identity + 0.15×script
+ [0.18 × spoofing_risk escalator]
Ratchet: rises ×1.5 fast · falls max 2.5 pts/s slow
Returns: named drivers (top 4) — every point is attributable
DIGITAL TWIN
twin.py
Markov chain
Fitted on 338
call corpus
Output:
"≈4m20s until
PAYMENT_EXEC"
LLM EXPLAINER
llm.py
Gemini Flash
Score+labels in
NEVER raw PII/audio
RAG: 31 chunks
BM25 + dense
Coach: 14 verbatim
human-reviewed lines
FRAUD INTELLIGENCE (FIGAE)
intel/
NetworkX graph
9 clusters / 114 cases
Community detection
Link prediction
Geospatial hotspots
AI investigation reports
(India gazetteer)
DATA PERSISTENCE
SQLite (default → Postgres via DATABASE_URL) + Vector Store + coach_library
Append-only audit log · Evidence packages (JSON+PDF) · Session state

DIAGRAM 02

The 7-Stage Scam Arc — State Machine

A digital arrest scam is a carefully orchestrated psychological arc. KAVACH models each stage and fires in the intervention window (Steps 2–5), approximately 10–15 minutes before a payment attempt.

1	2	3	4	5	6	7
GREETING	AUTHORITY CLAIM	FEAR INDUCTION	ISOLATION	VERIFICATION DEMAND	PAYMENT SETUP	PAYMENT EXECUTION

Stage	Caller's Goal	Victim Psychology	KAVACH Trigger
GREETING	Establish trust frame	Normal / unsuspecting	Monitor only
AUTHORITY_CLAIM	Assert authority	Startled / confused	Passport check fires
FEAR_INDUCTION	Create fear of legal action	Frightened / anxious	Coercion index rises
ISOLATION	Cut off support network	Isolated / dependent	🔴 CRITICAL coach fires
VERIFICATION_DEMAND	Extract PII + credentials	Compliant / panicked	🔴 Payment HOLD at 55+
PAYMENT_SETUP	Orchestrate transfer	Fully compliant	⛔ Guardian mode + circuit breaker
PAYMENT_EXECUTION	Complete fraud	Devastated	Evidence auto-generated
BENIGN	Legitimate institutional call	Normal / cooperative	CALM — false positive suppressed

DIAGRAM 03

Threat Fusion Engine — Signal Flow

Six independent signals are fused into a single 0–100 threat score. Four are weighted conversational signals; two are bounded escalators that add on top without diluting the core weights.

40%
Stage Probability
MuRIL/lexical 8-class distribution. Never a bare argmax.

25%
Coercion Index
Victim utterances ONLY. Independent of stage classifier.

20%
Identity Failure
CBI/Customs/Police SOPs checked mechanically. Citable source per FAIL.

15%
Script Similarity
Dense + lexical TF-cosine. Gate at 0.45 prevents coincidental matches.

```
THREAT FUSION ENGINE  —  threat.py
═══════════════════════════════════════════════════════════════════════

  INPUT SIGNALS                    WEIGHTS        NOTES
  ──────────────────────           ────────       ──────────────────────────
  Stage probability (classifier)   W = 0.40       discounted by confidence
  Manipulation pressure (cumulative) W = 0.25     rises only, never falls
  Coercion index (victim only)     W = 0.20       independent of classifier
  Identity trust failure           W = 0.15       only runs when passport checked
                                   ────────
                                   Σ = 1.00

  Escalators (add on top, do NOT replace signal weight):
  ──────────────────────────────────────────────────────
  Number spoofing risk             W = 0.18 escalator
    └─ Caller-ID mismatch, VoIP, intl routing, reported-number, call frequency
    └─ Capped: spoofing alone cannot reach CRITICAL on a silent call

  Script similarity                W = 0.15 escalator  (gate: ≥ 0.45)
    └─ Below 0.45 = zero contribution; shared common words never fires it

  ──────────────────────────────────────────────────────────────────────
  FORMULA:
    raw = (stage_component + tactic_component + coercion_component
           + trust_component + spoof_component + script_component)
    score = clamp(raw, 0, 100)

  RATCHET MECHANISM:
    floor = previous_score − DECAY_PER_S × dt     (DECAY_PER_S = 2.5 pts/s)
    if score < floor: score = floor

    → Score rises FAST (×1.5 per threatening turn)
    → Score falls SLOWLY (max 2.5 pts/s of decay)
    → A scammer who pauses to seem calm does NOT reset an active CRITICAL

  OUTPUT:
    FusionResult {
      score:   78.5,          # 0–100 fused threat score
      level:   "HIGH",        # CALM|WATCH|ELEVATED|HIGH|CRITICAL
      drivers: [              # Named contributors, top 4 sorted by contribution
        {label, contribution, detail},  # "every point is attributable"
      ]
    }

  THRESHOLD TABLE:
  ──────────────────────────────────────────────────────
    CALM      0–24     Monitor only
    WATCH     25–49    Advisory notice
    ELEVATED  50–69    Coach panel shown
    HIGH      70–89    Guardian mode fires
    CRITICAL  90–100   Full guardian + payment block
    ─────────
    Payment HOLD activates independently at score ≥ 55
```

Manipulation Accumulator — Tactic Pressure (Never Falls)

```
ManipulationAccumulator tracks cumulative tactic use across the call.
Unlike the score (which ratchets), tactic pressure NEVER falls —
the call does not become less manipulative because the manipulation stopped.

TACTIC BARS:              CHARGED BY:              CHARGE RATE (per utterance × confidence)
──────────────────        ──────────────────       ──────────────────────────────────────────
authority             ←   AUTHORITY_CLAIM stage    +0.34 × classifier_confidence
fear                  ←   FEAR_INDUCTION stage     +0.30 × classifier_confidence
isolation             ←   ISOLATION stage          +0.38 × classifier_confidence
urgency               ←   VERIFICATION_DEMAND      +0.22 × classifier_confidence
urgency               ←   PAYMENT_SETUP            +0.26 × classifier_confidence
compliance            ←   PAYMENT_EXECUTION        +0.40 × classifier_confidence
compliance  (bonus)   ←   VICTIM says COMPLIANT    +0.15 (victim behaviour evidence)
fear        (bonus)   ←   VICTIM shows PANICKED    +0.10 (victim behaviour evidence)

pressure = mean(authority, fear, isolation, urgency, compliance)  → 0–1
tactic_component = pressure × W_TACTIC × 100
```

DIAGRAM 04

ML Training Pipeline — 5 Phases (Fully Reproducible)

```
ML TRAINING PIPELINE  —  ml/ directory
═══════════════════════════════════════════════════════════════════════════

  PHASE 1 — CORPUS GENERATION                                ml/generate_calls.py
  ─────────────────────────────────────────────────────────────────────────
  Backend options:
    ├── Option A: Gemini free tier (Google AI Studio, no billing)
    └── Option B: Ollama local (ollama pull qwen2.5:7b-instruct — fully offline)

  Generates: 338 scam calls × 15–40 utterances each
  Archetypes: courier_scam · narcotics_arrest · customs_officer · cbi_arrest
              kyc_update · investment_fraud · refund_qr · bank_official
  Languages:  Hinglish (primary) · Hindi · code-switched English
  Format:     [{speaker: CALLER|VICTIM, text: "...", stage: STAGE_ENUM}]

  Augmentation: ml/paraphrase.py — adds paraphrases for lexical diversity
  Fallback:     ml/synth_seeds.py — deterministic no-LLM generator (when quota gone)

  PHASE 2 — DATASET BUILDING                                 ml/build_dataset.py
  ─────────────────────────────────────────────────────────────────────────
  → Schema validation (Pydantic) — rejects any malformed generation
  → Format: "CALLER: previous_utterance [SEP] CALLER: current_utterance"
            (speaker context is load-bearing for the model)
  → Split: LEAVE-ARCHETYPES-OUT BY CALL (strict — prevents memorisation)
            Train: 931 utterances  |  Val: 138 utterances  |  Test: 171 utterances
  → Builds: transition matrix (Stage(t) → Stage(t+1) with dwell times)
            Written to ml/data/transitions.json → loaded by twin.py (Digital Twin)

  PHASE 3 — MODEL TRAINING                                   ml/train.py
  ─────────────────────────────────────────────────────────────────────────
  Base:     google/muril-base-cased — multilingual Transformer (Hinglish-aware)
  Head:     8-way linear classifier over pooled [CLS] embedding
  Training: CrossEntropyLoss, AdamW, warmup_ratio=0.1, lr=2e-5
  Platform: CPU (⚠️ Apple MPS pins at ln(8)=2.079 loss — known torch 2.2 bug)
  Bug fix:  save_safetensors=False (MuRIL has non-contiguous tensors)
  Bug fix:  classification_report guard for missing classes in split
  Output:   ml/artifacts/checkpoint/ + config.json + label_ordering.json

  PHASE 4 — PROMOTION GATE                                   ml/eval_backends.py
  ─────────────────────────────────────────────────────────────────────────
  Measures both backends on the HELD-OUT TEST SET (archetypes never seen in train)
  Metric: macro-F1 (strict — all 8 classes weighted equally)

  Current measured result:

    ┌──────────────────────────┬──────────────┬──────────────────────────────┐
    │ Backend                  │ Val macro-F1 │ Test macro-F1 (held-out archetypes)│
    ├──────────────────────────┼──────────────┼──────────────────────────────┤
    │ Lexical baseline         │      —       │ 0.368  ← CURRENTLY WINS (serves)  │
    │ Fine-tuned MuRIL         │    0.983     │ 0.221  (overfits to archetypes)   │
    └──────────────────────────┴──────────────┴──────────────────────────────┘

    The 0.983 val F1 is memorisation — not generalisation.
    The split is telling the truth. The fix is more diverse training data.

  Written to: ml/artifacts/backend_comparison.json  (load-bearing — in git)

  PHASE 5 — AUTOMATIC SERVING                                engine/classifier.py
  ─────────────────────────────────────────────────────────────────────────
  load_classifier() reads backend_comparison.json at startup
  → Auto-promotes the measured winner (currently: lexical)
  → Reports honestly: "lexical · best" (not a fault — the promotion gate working)
  → Force override: PRESAGE_CLASSIFIER=muril (for testing checkpoint)
  → Pre-warmed at @app.on_event("startup") — zero cold-start on first request
```

Digital Twin — Markov-Chain Forecaster

```
twin.py — fitted on stage transition frequencies from build_dataset.py

TRANSITION MATRIX (example, simplified):
──────────────────────────────────────────────────────────────────────
From Stage           → To ISOLATION  → To VERIF_DEMAND  → To PAYMENT_SETUP
GREETING                  0.02            0.00              0.00
AUTHORITY_CLAIM           0.41            0.12              0.02
FEAR_INDUCTION            0.38            0.19              0.03
ISOLATION                 0.15            0.53              0.08
VERIFICATION_DEMAND       0.01            0.12              0.59

WITH DWELL TIMES per stage (mean seconds):
GREETING: 45s · AUTHORITY_CLAIM: 90s · FEAR_INDUCTION: 180s
ISOLATION: 120s · VERIFICATION_DEMAND: 240s · PAYMENT_SETUP: 180s

FORECAST QUERY at ISOLATION stage:
→ Path: ISOLATION → VERIFICATION_DEMAND → PAYMENT_SETUP → PAYMENT_EXECUTION
→ Expected time: 120 + 240 + 180 = 540s = "≈9 minutes"
→ Output: {minutes_to_payment: 4.3, confidence: 0.72, path: [...]}
```

DIAGRAM 05

Live Protection Session — Sequence Diagram

STATEFRAME STRUCTURE (WEBSOCKET PAYLOAD EVERY 250MS)

{"session_id":"abc123","score":85.0,"level":"CRITICAL","stage":"ISOLATION","stage_confidence":0.87,"coercion_index":34,"victim_state":"FEARFUL","payment_state":
[{"label":"Stage: Isolation","contribution":0.364},...],"coach_line":"Hang up. Real police never...","twin_forecast":{"minutes_to_payment":4.3}}

DIAGRAM 06

Module 2 — FIGAE Fraud Intelligence Graph

```
FIGAE — FRAUD INTELLIGENCE GRAPH & ANALYTICS ENGINE
═══════════════════════════════════════════════════════════════════════

  NODE TYPES                          EDGE TYPES
  ──────────────────────────          ──────────────────────────────────
  📞 Phone Number                     ── shared_call     appeared in same incident
  💳 UPI VPA (Virtual Payment Addr)   ── shared_tx       financial transaction link
  🏦 Bank Account + IFSC              ── co_cluster      community detection edge
  📱 Device Fingerprint               ── predicted_link  graph AI inference (hidden)
  🗺️  Geographic Location              ── geo_proximity   hotspot cluster membership

  GRAPH STATISTICS (current build):
  ──────────────────────────────────
    Total seeded cases:    114 historical incidents
    Detected campaigns:    9 fraud clusters (community detection)
    Risk distribution:     LOW: 2 · MEDIUM: 3 · HIGH: 3 · CRITICAL: 1
    Kingpin identification: betweenness centrality on directed graph

  ──────────────────────────────────────────────────────────────────────
  INGEST PIPELINE — how new cases enter the graph:

  Module 1 Session ──► Entity Extraction (analyzer.py) ──► Graph Ingest
                       │ Extracts automatically from transcript:
                       │ • Phone numbers    (E.164 format, ISD prefix detection)
                       │ • UPI VPAs         (user@psp, regex validated)
                       │ • Bank accounts    (9–18 digit patterns + IFSC)
                       │ • Device signals   (caller-ID metadata from spoofing)
                       └─► Creates/updates nodes → adds edges → re-clusters

  COMMUNITY DETECTION:
  ──────────────────────────────────────────────────────────────────────
    Algorithm:  Louvain (modularity-maximising) / connected components fallback
    Input:      NetworkX undirected projection (phone + UPI nodes, shared_call edges)
    Output:     Partition map → 9 clusters from 114 cases
    Re-runs:    On every new case ingest (dynamic, not batch)

  CLUSTER RISK SCORING:
  ──────────────────────────────────────────────────────────────────────
    risk = f(cluster_size, max_individual_loss, victim_density, recency, betweenness)
    → LOW (monitoring) · MEDIUM (watch) · HIGH (action required) · CRITICAL (immediate)
    Updates dynamically when new cases added

  LINK PREDICTION (Graph AI):
  ──────────────────────────────────────────────────────────────────────
    Uses Jaccard coefficient / common-neighbour heuristics on the bipartite graph
    Surfaces: "Phone A and Phone B likely share infrastructure despite no direct link"
    Useful for: identifying burner SIM rings buying numbers from same vendor

  GEOSPATIAL HOTSPOTS:
  ──────────────────────────────────────────────────────────────────────
    India gazetteer (district-level coordinates for all 640+ districts)
    Clustering: spatial proximity grouping of case origin metadata
    Output: {lat, lon, intensity, cluster_id} — consumed by react-leaflet
    Purpose: inter-district law enforcement intelligence sharing

  AI INVESTIGATION REPORTS:
  ──────────────────────────────────────────────────────────────────────
    Per-cluster · auto-generated via RAG + Gemini
    Template: cluster summary → key nodes → evidence timeline → victim list → recommendations
    FC-001 format reproduces the FC-021 exemplar from the PS6 problem brief
```

API Endpoint	Method	Returns
`/api/intel/graph`	GET	All nodes + edges (force-directed vis data)
`/api/intel/clusters`	GET	9 clusters with risk, size, key members
`/api/intel/map`	GET	GeoJSON hotspot data for react-leaflet
`/api/cases`	GET/POST	Case book — save/retrieve evidence (Auth: Analyst+)

DIAGRAM 07

Security Architecture — Layers & Privacy by Design

Control	Implementation	Standard	Status
Rate limiting	Token-bucket in security.py	OWASP API Security	✅
Brute-force	Login backoff (exponential delay)	CWE-307	✅
CSP headers	4 hardening response headers	OWASP ASVS	✅
CORS lockdown	Strict single-origin whitelist	OWASP Top 10	✅
Auth algorithm	pbkdf2_hmac + HS256 (stdlib)	NIST SP 800-57	✅
Secret management	Env vars only — zero secrets in git	NIST SP 800-57	✅
Audio privacy	Whisper LOCAL — never to cloud	GDPR / Privacy by design	✅
LLM isolation	Score+labels only — no PII to Gemini	Data minimisation	✅

DIAGRAM 08

Frontend Route Architecture

```
FRONTEND ROUTE ARCHITECTURE — apps/web/src/
═══════════════════════════════════════════════════════════════════════════

  APP SHELL (AppShell.tsx)
  ─────────────────────────────────────────────────────────────────────────
  • ⌘K  CommandPalette — keyboard-driven navigation to any route
  • 🌙  Light / Dark theming — system-preference aware, persisted in localStorage
  • 🔒  RBAC-scoped navigation — menu items shown/hidden by role
  • 🔄  RouteBoundary — error boundary + suspense for lazy-loaded routes
  • 📡  useHealth() hook — polls /api/health, shows live degraded badges

  ─────────────────────────────────────────────────────────────────────────
  PUBLIC ROUTES (no authentication required)
  ─────────────────────────────────────────────────────────────────────────

  /           Home.tsx            WebGL Three.js hero + GSAP entrance animation
              ──────────────      "Awwwards-style" landing outside the app shell
              Hooks: none          Direct link: Login / Try Demo

  /login      Login.tsx           3D tilt card (CSS perspective transform)
              ──────────────      Seeded role roster (owner/admin/analyst/viewer)
              Auth: AuthContext    Open demo mode: any credentials accepted
                                  POST /api/auth/login → stores JWT in context

  /shield     Shield.tsx          Citizen Fraud Shield (Module 3 — CFSRP)
              ──────────────      Public, no login required
              Public API:          Threat verification + stage-aware coaching
              /api/shield/*        Emergency helplines + complaint PDF generator

  ─────────────────────────────────────────────────────────────────────────
  PROTECTED ROUTES (authentication required)
  ─────────────────────────────────────────────────────────────────────────

  /dashboard  Dashboard.tsx       System health overview
              Min role: Viewer     Active sessions count · API status badges
              Hooks: useHealth     Module status (RSSIE / FIGAE / CFSRP)

  /console    LiveProtection.tsx  🔴 LIVE PROTECTION COCKPIT
              Min role: Viewer     Real-time Threat Meter (0–100 animated)
              Hooks: useLiveSession Coach panel (verbatim lines)
                     useStreamPlayer WebSocket 4 Hz StateFrame render
                                  Digital Twin forecast ("≈4m20s")
                                  Mock stream replay when API offline

  /guardian   Guardian.tsx        Evidence vault
              Min role: Analyst    Save session to case book
              Hooks: session API   Export evidence PDF
                                  View/share token-addressed CitizenReport

  /analyzer   Analyzer.tsx        Stateless single-utterance analysis
              Min role: Analyst    Text input + image upload (OCR)
              POST /api/analyze    Immediate full engine pipeline result
              POST /analyze/image  Named drivers + scores displayed

  /knowledge  Knowledge.tsx       RAG knowledge assistant
              Min role: Analyst    Ask questions → cited corpus answers
              POST /knowledge/ask  "How do I verify CBI calls?" → sourced response

  /intel      Intel.tsx           Fraud intelligence (Module 2 — FIGAE)
              Min role: Analyst    Force-directed fraud graph (react-force-graph)
              /api/intel/*         India scam map (react-leaflet)
                                   Cluster list with risk badges
                                   AI investigation report per cluster

  /model      ModelCard.tsx       ML model card + backend comparison
              Min role: Analyst    Shows backend_comparison.json metrics
                                   MuRIL vs lexical — honest F1 display
                                   Training pipeline documentation

  /cases      CaseBook.tsx        Case book — Admin+ only
              Min role: Admin      All saved cases with filter/search
              /api/cases           Audit log: logins, exports, overrides
                                   User roster (Admin: create/disable users)

  ─────────────────────────────────────────────────────────────────────────
  KEY DESIGN: ZERO THREAT LOGIC IN REACT
  ─────────────────────────────────────────────────────────────────────────

  Every number the UI displays is a contract field from the API.
  No threshold comparisons, no stage string matching, no score calculations
```

```
exist in any React component. The UI is a pure renderer of the engine's output.
TypeScript typecheck enforces this — undefined contract fields are caught at build.
```

DIAGRAM 09

Schema Contract Architecture — Single Source of Truth

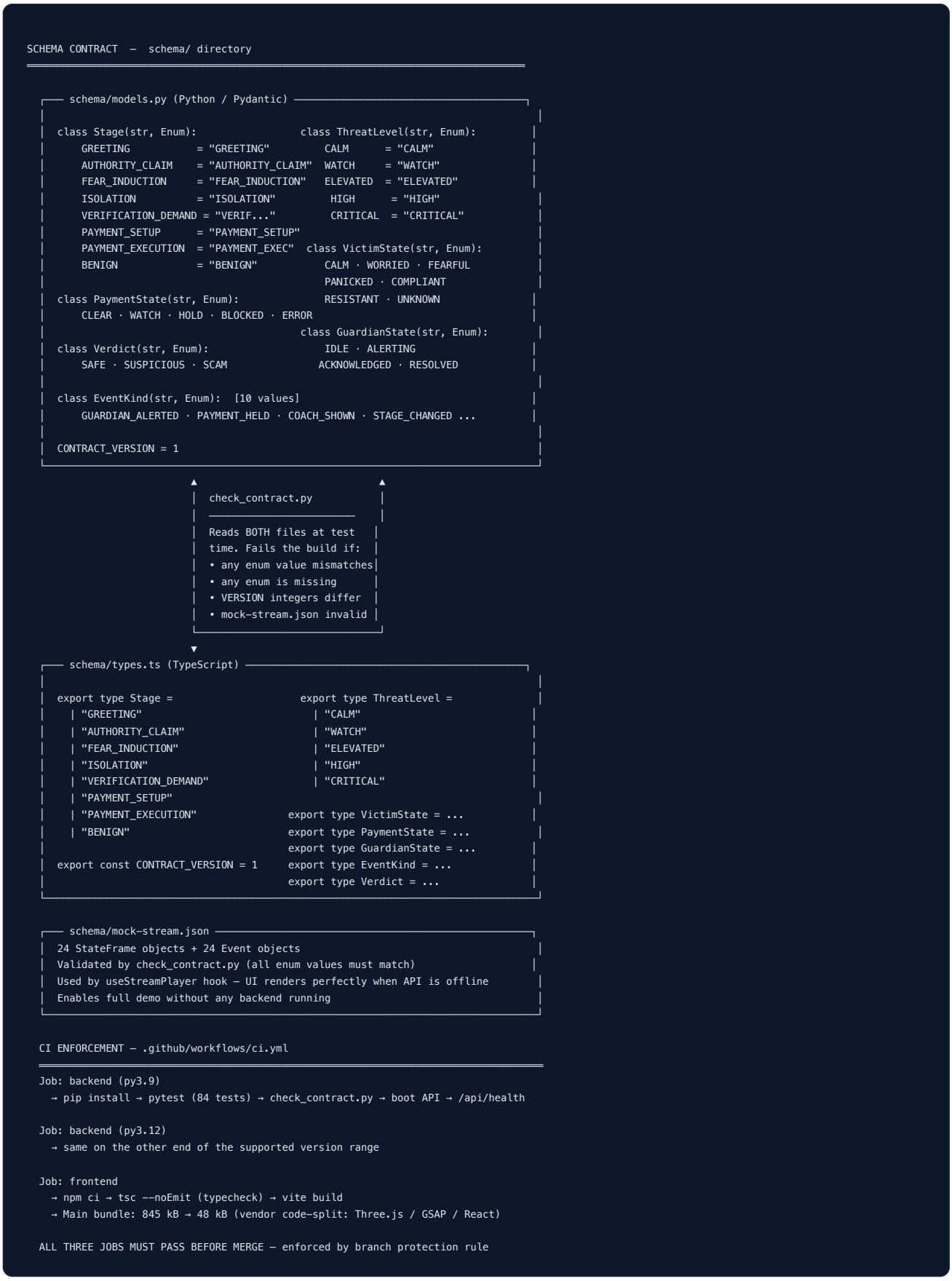
SCHEMA CONTRACT — schema/ directory
schema/models.py (Python / Pydantic)
class Stage(str, Enum):
GREETING = "GREETING"
AUTHORITY_CLAIM = "AUTHORITY_CLAIM"
FEAR_INDUCTION = "FEAR_INDUCTION"
ISOLATION = "ISOLATION"
VERIFICATION_DEMAND = "VERIF..."
PAYMENT_SETUP = "PAYMENT_SETUP"
PAYMENT_EXECUTION = "PAYMENT_EXEC"
BENIGN = "BENIGN"
class ThreatLevel(str, Enum):
CALM = "CALM"
WATCH = "WATCH"
ELEVATED = "ELEVATED"
HIGH = "HIGH"
CRITICAL = "CRITICAL"
class VictimState(str, Enum):
CALM · WORRIED · FEARFUL
PANICKED · COMPLIANT
class PaymentState(str, Enum):
CLEAR · WATCH · HOLD · BLOCKED · ERROR
class GuardianState(str, Enum):
IDLE · ALERTING
ACKNOWLEDGED · RESOLVED
class Verdict(str, Enum):
SAFE · SUSPICIOUS · SCAM
class EventKind(str, Enum): [10 values]
GUARDIAN_ALERTED · PAYMENT_HELD · COACH_SHOWN · STAGE_CHANGED ...
CONTRACT_VERSION = 1
check_contract.py
Reads BOTH files at test time. Fails the build if:
• any enum value mismatches
• any enum is missing
• VERSION integers differ
• mock-stream.json invalid
schema/types.ts (TypeScript)
export type Stage =
| "GREETING"
| "AUTHORITY_CLAIM"
| "FEAR_INDUCTION"
| "ISOLATION"
| "VERIFICATION_DEMAND"
| "PAYMENT_SETUP"
| "PAYMENT_EXECUTION"
| "BENIGN"
export const CONTRACT_VERSION = 1
export type ThreatLevel =
| "CALM"
| "WATCH"
| "ELEVATED"
| "HIGH"
| "CRITICAL"
export type VictimState = ...
export type PaymentState = ...
export type GuardianState = ...
export type EventKind = ...
export type Verdict = ...
schema/mock-stream.json
24 StateFrame objects + 24 Event objects
Validated by check_contract.py (all enum values must match)
Used by useStreamPlayer hook — UI renders perfectly when API is offline
Enables full demo without any backend running
CI ENFORCEMENT — .github/workflows/ci.yml
Job: backend (py3.9)
→ pip install → pytest (84 tests) → check_contract.py → boot API → /api/health
Job: backend (py3.12)
→ same on the other end of the supported version range
Job: frontend
→ npm ci → tsc --noEmit (typecheck) → vite build
→ Main bundle: 845 kB → 48 kB (vendor code-split: Three.js / GSAP / React)
ALL THREE JOBS MUST PASS BEFORE MERGE — enforced by branch protection rule

DIAGRAM 10

Scalability Path — Hackathon to National Scale

```
SCALABILITY ARCHITECTURE
═══════════════════════════════════════════════════════════════════════════

  CURRENT STATE (Hackathon Build)          PRODUCTION SWAP (One env var)
  ──────────────────────────────────       ──────────────────────────────────

  SQLite (single file)                →    PostgreSQL  (DATABASE_URL=postgresql://)
  NetworkX (in-memory graph)          →    Neo4j       (same interface — Cypher)
  Local Whisper (faster-whisper)      →    Sarvam ASR  (SARVAM_API_KEY=)
  Single uvicorn instance             →    Multiple workers (stateless /analyze)
  Dev CORS (localhost only)           →    Production CORS list
  Ephemeral sessions (in-memory)      →    Persistent (DATABASE_URL set)

  ALL SWAPS ARE ONE ENV VAR OR ONE ADAPTER — no code changes to the engine

  ──────────────────────────────────────────────────────────────────────────
  PHASE 1 — APP LEVEL (Current)
  ──────────────────────────────────────────────────────────────────────────
  Target:    Individual citizens with smartphones
  Access:    Web browser → http://localhost:5173 / production URL
  Scale:     Single FastAPI instance · hundreds of concurrent WebSocket sessions
  ASR:       Local Whisper (mock stream for demo)
  Storage:   SQLite persistent file (kavach.db)
  Auth:      RBAC seeded with demo roles

  ──────────────────────────────────────────────────────────────────────────
  PHASE 2 — IVR INTEGRATION (Post-hackathon)
  ──────────────────────────────────────────────────────────────────────────
  Target:    Feature-phone users · rural India (no smartphones)
  Access:    PSTN telephone call → IVR bridge → KAVACH WebSocket stream
  Scale:     Horizontal scaling of stateless /api/analyze endpoint
  ASR:       Sarvam ASR (SARVAM_API_KEY= swap, same interface)
             Supports 12 Indian languages including regional dialects
  Storage:   PostgreSQL (DATABASE_URL= swap)
  Integration: TRAI-registered IVR service provider

  ──────────────────────────────────────────────────────────────────────────
  PHASE 3 — TELECOM PROVIDER LEVEL (Q1 2027)
  ──────────────────────────────────────────────────────────────────────────
  Target:    1.2 billion subscribers (all Indian mobile users)
  Access:    TRAI/DoT partnership — detection at network/SIM layer
             Call metadata + (opt-in) audio → KAVACH inference cluster
  Scale:     Kubernetes cluster · load-balanced inference · Redis session store
  Graph:     Neo4j cluster (NetworkX → Neo4j same interface, already designed)
  Coverage:  All incoming calls flagged with risk score before connecting

  ──────────────────────────────────────────────────────────────────────────
  PHASE 4 — NATIONAL PLATFORM (2027+)
  ──────────────────────────────────────────────────────────────────────────
  Target:    Complete national coverage
  NCRB:      Automated 1930 case filing (structured JSON → NCRB API)
  MHA:       Real-time aggregate analytics → monthly threat landscape reports
  WhatsApp:  WhatsApp Business API → send coach lines via chat during call
  Regions:   12 regional language models (Sarvam covers: Hindi, Tamil, Telugu,
             Kannada, Malayalam, Bengali, Gujarati, Marathi, Punjabi, Odia,
             Rajasthani, Assamese)
  Expansion: Counterfeit currency module (image classification of currency notes)
             Investment fraud detection (multi-session relationship tracking)

  ──────────────────────────────────────────────────────────────────────────
  STATELESS ANALYSIS PATH — key to horizontal scaling
  ──────────────────────────────────────────────────────────────────────────
  POST /api/analyze  →  no session state  →  pure function(utterance) → result
  This endpoint is already stateless and can scale to any number of workers.
  The WebSocket path (live sessions) uses per-session state in the process —
  scaling this requires Redis session store or sticky load balancing, both standard.
```

Scale Level	Citizen Reach	Key Tech Change	Timeline
App (Done)	Smartphone users	None — runs now	✅ Complete
IVR Integration	Feature phones + rural	Sarvam ASR + PSTN bridge	Post-hackathon
Telecom Provider	1.2 billion subscribers	TRAI/DoT API + Kubernetes	Q1 2027
National Platform	Complete coverage	NCRB + WhatsApp + 12 languages	2027+